

ARBITER — Adaptive Multi-Agent Orchestration under Matched Compute Budgets

Phase 0 Planning Package — 7th Semester B.Tech CSE Project, SMVDU Katra

Team	Gautam Lasgotra (23BCS032) · Chirag Attri (23BCS024) · Aniket Kundal (23BCS015)
Mentor	Faculty mentor, SoCSE, SMVDU
Date	24 September 2026
Status	PLANNING — no implementation authorised
Supersedes	VYAPARI (retail/inventory) — fully discarded

Context — why this project, and why this shape

The mentor rejected Vyapari because a 3-person semester project could not be justified against an already crowded commercial category. The new direction is agentic AI: agents that produce work, other agents that validate it, feedback loops, iteration until acceptance criteria are met — with **research as the primary objective**.

The trap in that brief is \$120: the proposed hypothesis ("adaptive task-specific workflow construction") is assumed to be a promising gap. **It is not a gap. It is a crowded, surveyed subfield.** The literature check below settles that. What the literature *does* leave open is something better: a question that is genuinely unresolved, cheaply measurable in seven weeks, and — critically — **yields a valid result even if the answer is "no."** That property is what makes this safe to attempt against a hard November deadline.

The single most important thing to internalise: on this project the code is the easy part. The core system is ~900–1300 lines of Python. The difficulty and the marks live in the benchmark, the runs and the numbers. This is the exact inverse of Vyapari.

PART A — RESEARCH POSITIONING

A1. Literature review matrix

(§8 of the brief; Aniket extends this to full paper-by-paper notes — this is the decision-relevant core)

Work	Architecture	Fixed/dynamic	Feedback	Benchmark	Main result	Limitation / what's open
Self-Refine (2023)	Single model, self-feedback	Fixed	Self-critique, no external signal	Multiple gen tasks	Iterative self-feedback improves output	No independent verifier; later contested
Reflexion (2023)	Agent + verbal reflection + episodic memory	Fixed	Verbal self-reflection + env signal	ALFWorld, HumanEval	Reflection improves success	Needs an env reward; memory unbounded
AgentCoder (2023)	Programmer + test designer + test executor	Fixed 3 roles	Executed unit tests	HumanEval, MBPP	Multi-agent + real tests beats single	Roles fixed; no budget control
ChatDev / MetaGPT (2023–24)	Role-play SW company / SOP-driven roles	Fixed	Inter-role review	SW tasks	Role specialisation produces fuller artifacts	Heavy token cost; weak objective grading
AutoGen (2023–)	Conversable multi-agent framework	Config-fixed	Message passing	Various	General orchestration substrate	Maintenance mode ; MS points to Agent Framework
Huang et al., "LLMs Cannot Self-Correct Reasoning Yet" (ICLR 2024)	—	—	Intrinsic self-correction	GSM8K, CQA, etc.	Self-correction without external feedback does not help and often degrades	Establishes that the <i>verifier</i> is the active ingredient
"When Can LLMs Actually Correct Their Own Mistakes?" (TACL)	Critical survey	—	—	—	Self-correction gains mostly come from external signals	Confirms above

Work	Architecture	Fixed/dynamic	Feedback	Benchmark	Main result	Limitation / what's open
MAST — "Why Do Multi-Agent LLM Systems Fail?" (NeurIPS 2025)	Taxonomy over 1600+ traces, 7 frameworks	—	—	—	14 failure modes / 3 categories ; step repetition 17.1%, reasoning-action mismatch 14.0%; most failures are <i>design</i> , not model	Gives us a ready-made error taxonomy
Tran & Kiela (2025) + OneFlow follow-ups	Budget-matched comparison	—	—	Multi-hop reasoning	At fixed thinking-token budget, single agent matches or beats multi-agent ; claimed advantages "better explained by unaccounted computation"	The deflationary result our design must answer
Wang et al. (matched-spend study)	Priced strategies in tokens	—	—	Reasoning	Elaborate strategies keep little advantage over plain self-consistency at equal spend	Same direction
DyFlow (2509.26062)	Designer + executor, runtime workflow gen	Dynamic	Intermediate feedback	Reasoning	Dynamic workflows adapt to task	Not budget-matched
DAAO (2509.11079)	Difficulty-aware depth/operator/model selection	Dynamic	—	Mixed	Difficulty-conditioned orchestration helps	Not budget-matched
AdaptOrch (2602.16873)	Selects parallel/sequential/hierarchical topology	Dynamic	—	Mixed	Task-adaptive topology helps	Not budget-matched
AgentSpawn (2602.07072)	Runtime spawning, static graph → dynamic tree	Dynamic	—	Long-horizon code	Spawning helps long-horizon work	Not budget-matched
MACA (2605.25746)	Budget-conditioned structural prior	Dynamic	—	Mixed	Learns participation structure	Learned, heavy; not a controlled A/B

Work	Architecture	Fixed/dynamic	Feedback	Benchmark	Main result	Limitation / what's open
AORCHESTRA (2602.03786)	On-demand subagents (instruction, context, tools, model)	Dynamic	Tool feedback	GAIA, SWE- Bench, Terminal- Bench	+16.28% over strongest baseline (Gemini-3- Flash)	Strong claim — but against best baseline, not matched budget
Survey: "From Static Templates to Dynamic Runtime Graphs" (2603.22386)	Survey of workflow optimisation	—	—	—	The subfield is mapped and active	Confirms adaptivity is not novel

A2. Prior-art conclusion and the actual gap

Adaptive orchestration is not novel. There is a survey of it. Any novelty claim on the architecture gets destroyed in viva. We will not make one.

What the literature contains instead is an **unresolved contradiction**:

- Budget-matched studies (Tran & Kiela, Wang et al.) show multi-agent advantages largely evaporate on **multi-hop reasoning** once tokens are held equal.
- Adaptive-orchestration papers (DyFlow, DAAO, AdaptOrch, AgentSpawn, AORCHESTRA) report solid gains — but evaluate against *strongest baseline*, **not against a compute-matched single agent**.

Both can be true simultaneously if the benefit depends on **task type** — specifically on whether a cheap, trustworthy external verifier exists. Huang et al. supply the mechanism: refinement works when there is external feedback and fails when there isn't.

The gap we occupy

*Adaptive orchestration's entire justification is "spend agents only when they are needed." That is a claim about compute efficiency. **It has therefore never been tested by the one protocol that directly measures compute efficiency — budget matching.** We run that test, and we run it across task families that differ in verifier strength.*

Why this is the right gap for this team: it is a measurement contribution, not an invention. It needs no novel algorithm. And a **negative result is publishable-quality and viva-defensible** — "we built it, matched the budget, and adaptivity did not pay off on family X" is a finding. That de-risks a seven-week deadline more than any other framing available.

A3. Research questions (narrowed to what is measurable by 10 Nov)

- **RQ1 (primary).** Under a matched token budget, does adaptive orchestration improve task correctness over (a) a budget-matched single agent and (b) fixed multi-agent pipelines?
- **RQ2.** Does any benefit depend on **verifier strength** (executable tests → execution match → answer-only)?
- **RQ3.** Does heterogeneous validation (tool + LLM critic) reduce **false accepts** versus LLM-only critique?
- **RQ4.** How many refinement iterations remain useful before returns vanish?

Dropped from the brief's list: RQ2-as-written (role specialisation in isolation) and RQ3-as-written (agent creation) are absorbed into RQ1; measuring them separately needs more conditions than the quota allows.

A4. Hypotheses — predictions, not results

- H1. At matched budget, adaptive $\approx$ budget-matched single agent on **math**; adaptive $>$ single on **code** and **SQL**.
- H2. Effect size increases monotonically with verifier strength.
- H3. LLM-only validation shows a materially higher false-accept rate than execution-based validation.
- H4. Most gain lands in iterations 1–2; iteration ≥ 3 contributes little.

No numbers are asserted anywhere in this document. All results come from logged runs.

PART B — WHAT WE ACTUALLY BUILD

B1. Definitions (§7 of the brief — these go in the report verbatim)

Term	Definition in this project
Agent	A role prompt + one LLM call + a Pydantic-typed parse of the result. Nothing more. Not a process, not a thread.
Tool	A deterministic, non-LLM function whose output is ground truth (pytest runner, SQL executor, answer matcher).
Validator	Any component producing a PASS/FAIL + evidence. May be a tool (objective) or an LLM critic (subjective).
Critic	An LLM validator that produces natural-language feedback rather than a binary signal.
Orchestrator	The <code>while</code> loop that owns state, invokes agents, applies the decision policy, enforces budget and stops.
Workflow	An ordered list of roles selected for one task, produced by the planner.
Iteration	One pass of generate → validate → decide.
Memory	The bounded feedback history carried in state. No vector DB, no episodic store.
Budget	Hard caps on tokens, LLM calls, wall time and iterations, enforced centrally.

B2. How this is actually built — the honest mechanics

An agent is a function:

```
class GeneratorOut(BaseModel):
    code: str
    notes: str

def generator(task: str, feedback: str | None, ctx: RunState) -> GeneratorOut:
    prompt = GEN_TEMPLATE.format(task=task, feedback=feedback or "none")
    raw = ctx.llm.call(prompt, schema=GeneratorOut) # provider abstraction + cache
    return GeneratorOut.model_validate_json(raw)
```

The orchestrator is a loop over a state object:

```
state = RunState(task=task, budget=Budget(max_iters=5, max_calls=20, max_tokens=60_000))
spec = planner.plan(state) # LLM call → which roles for THIS task
while not state.should_stop():
    out = run_role(spec.generator, state)
    ev = validate(out, spec.validators, state) # tools first, LLM critic second
    action = decide(ev, state) # ACCEPT | REFINE | ADD_AGENT | ESCALATE | STOP
    state.record(spec, out, ev, action)
    if action is ACCEPT: break
    if action is ADD_AGENT: spec = spec.with_role(pick_role(ev))
    state.feedback = summarise(ev)
```

Validation for code contains no LLM at all:

```
subprocess.run([sys.executable, "-c", prog], timeout=5, capture_output=True, env=SAFE_ENV)
```

That subprocess is the external feedback Huang et al. identify as the active ingredient. The trace is a list of dicts appended every step and dumped to JSONL; every metric in the paper is computed from it. There is no separate metrics system.

"Adaptive" means `planner.plan()` returns a different role list per task. That is the whole mechanism. It is deliberately small so it can be inspected, explained and defended.

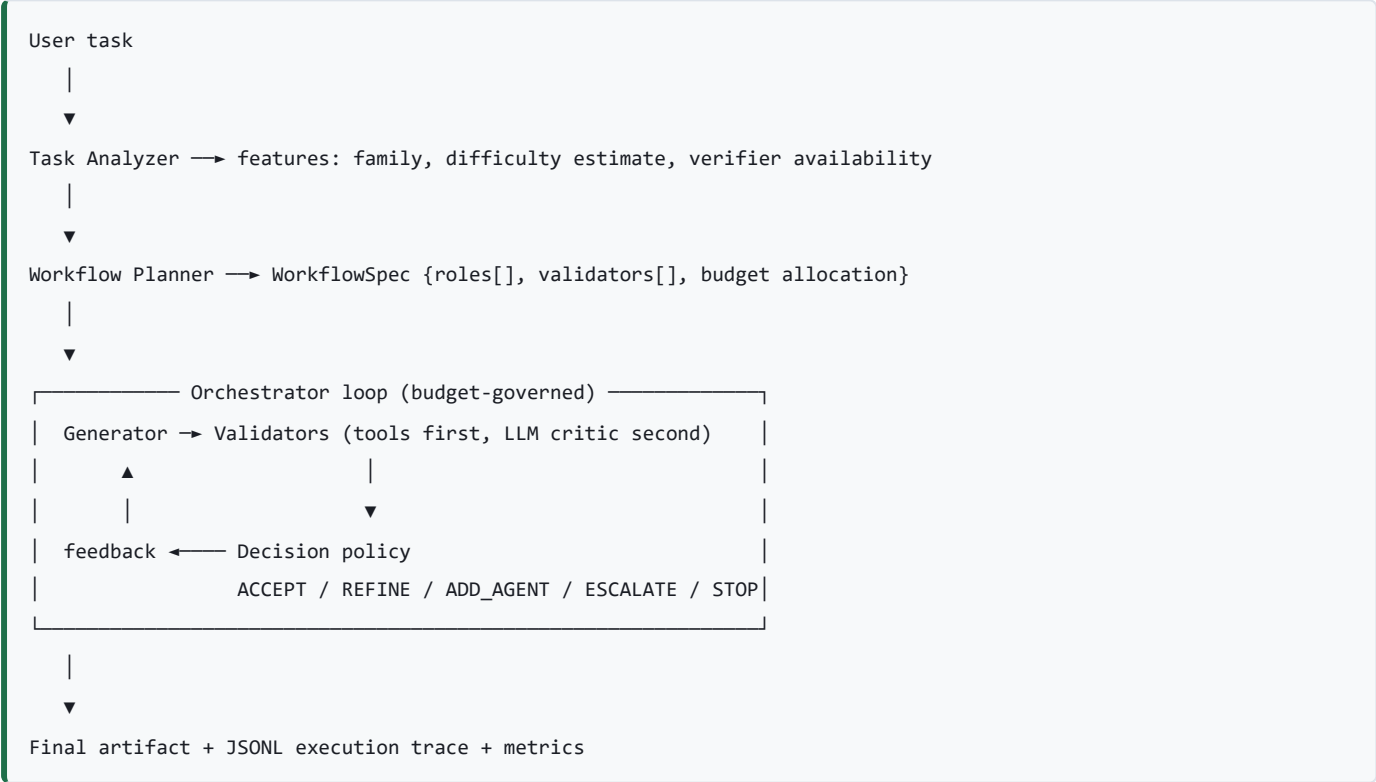
B3. Critical design decision: selection, not invention

The brief flirts with agents being *created* at runtime. We will not do that.

- **Rejected:** free-form runtime agent invention. Unbounded, unreproducible, impossible to evaluate, and a direct route to the MAST failure modes (step repetition, reasoning-action mismatch).
- **Chosen:** a fixed role library — `planner, generator, tester, reviewer, sql_specialist, math_checker, repair` — from which the planner *selects* a subset per task, and from which the decision policy may *add* one mid-run.

Dynamic **selection** from a known library is adaptive, measurable and defensible. Dynamic **invention** is a demo, not an experiment. State this trade-off explicitly in the report — it is exactly the kind of reasoning a viva rewards.

B4. Architecture



The Budget Governor is a first-class, cross-cutting component, not an afterthought. It meters every call and is what makes budget-matched comparison possible at all. It is the methodological heart of the project.

B5. Conditions (baselines) — the experimental design

ID	Condition	Description
A	Single agent	One call, one shot. Conventional baseline.
A+	Budget-matched single agent	Same token budget as D, spent on self-consistency / best-of-N. The critical baseline.
B	Fixed two-agent loop	Generator ↔ validator, fixed iterations.

ID	Condition	Description
C	Fixed pipeline	Planner → generator → tester → reviewer.
D	ARBITER (adaptive)	Analyzer → planner → dynamic roles → iterative refinement → budget-aware termination.

A+ is what makes this project credible. Nearly every student multi-agent project omits it and therefore reports a meaningless win. Including it is the contribution.

B6. Benchmark

Family	Source	Verifier	Verifier strength	n
Code	HumanEval+ / MBPP+ (EvalPlus)	pytest execution in sandbox	Strong (objective, per-test)	40
SQL	Spider dev subset	execution-result match vs gold on SQLite	Medium (objective, final only)	40
Math	GSM8K subset	exact final-answer match	Weak (no usable intermediate signal)	40

120 tasks total. Frozen before final runs (\$255) and committed with a hash. Contamination is a known limitation for HumanEval/GSM8K — stated in threats to validity, not hidden.

This axis is the experiment. It is chosen so RQ2 is answerable with three points on a line rather than a single anecdote.

B7. Metrics (all derived from the trace)

Correctness: pass@1, per-test pass rate, execution-match rate. Cost: prompt/completion tokens, LLM calls, agents used, iterations, wall latency, estimated ₹ cost. Quality of process: false-accept rate (validator says PASS, held-out test says FAIL) — the key number for RQ3; convergence/non-convergence; iteration-of-first-success. Failures: classified using the MAST taxonomy (existing, citable — Aniket does not invent one).

B8. Termination policy (\$9 — never unbounded)

Stop on any of: validator PASS + reviewer ACCEPT · max 5 iterations · max 20 LLM calls · max 60k tokens · max 180s wall · no score improvement for 2 iterations · identical-state cycle detected (hash of output). Every stop reason is logged as an enum.

B9. Safe execution (\$10 — non-negotiable)

- Local research runner: subprocess, no network, temp cwd, RLIMIT_CPU / RLIMIT_AS , 5s timeout, restricted env. Docker optional if time allows.
- Public demo on Render: no arbitrary Python execution. Live demo serves SQL (read-only SQLite) and math families. Code-family runs are replayed from stored traces, clearly labelled as a replay.

This satisfies the brief's security rule without losing the demo, and "we deliberately did not expose RCE on a public host" is a good viva answer.

PART C — ENGINEERING

C1. Stack (final)

Layer	Choice	Why
Language	Python 3.12+	Ecosystem, as briefed
API	FastAPI + SSE	Live agent events to the UI without websockets complexity
Models/validation	Pydantic v2	Structured LLM outputs, typed state
Orchestration	Custom, ~400 lines	The contribution must be visible, not buried in LangGraph
LLM	Gemini Flash-Lite tier via provider abstraction	Best free quota; verify exact model/limits at implementation
2nd provider	Groq / OpenRouter free	Needed for the different-model-reviewer comparison (\$227) and doubles quota
Storage	SQLite + JSONL traces	No cloud DB needed
Frontend	Server-rendered HTML + htmx/vanilla JS	Brief says don't over-invest; SSE stream drives agent cards
Deploy	Render free	Public HTTPS for the panel

Not used: AutoGen (maintenance mode), LangGraph (hides the contribution), any vector DB, any paid cloud.

C2. Quota strategy — the biggest practical risk

Free-tier reality: Flash-Lite $\approx$ 15 RPM / $\sim$ 500 RPD; larger Flash models are far tighter. A full sweep is roughly **4 conditions $\times$ 120 tasks $\times$ 2 seeds $\approx$ 960 task-runs $\approx$ 6,000+ LLM calls**. On one free account that is weeks. Four controls make it work on near-zero spend:

- 1. Response cache, day one.** Key = `sha256(provider, model, prompt, temperature, seed)`. Re-runs and report regeneration become free, and it makes runs reproducible — which \$292 requires anyway. This single component is the difference between feasible and not.
- 2. Three accounts, one per member,** round-robin by the provider abstraction $\rightarrow$ $\sim$ 1,500 RPD. A full sweep lands in $\sim$ 4 days of wall clock.
- 3. Resumable experiment runner.** Checkpoint after every task. A quota error must never destroy a six-hour run. Non-negotiable.
- 4. A dedicated demo API key that the experiment runner never touches,** so RPD is guaranteed fresh on evaluation day.

Demo safety net: the UI supports replaying a stored trace. If the API fails in front of the panel, the demo still runs. Build this in Phase 1, not the night before.

Optional insurance: Flash-Lite paid rates are very low, so a small top-up would remove residual risk entirely. Verify current pricing before deciding — do not assume figures.

C3. Repository structure

```
arbitrator/  
├─ llm/          provider.py  gemini.py  groq.py  cache.py  accounting.py  
├─ agents/       roles.py    prompts/  schemas.py  
├─ orchestrator/ state.py    planner.py analyzer.py policy.py  budget.py  loop.py  
├─ validators/   python_exec.py sql_exec.py answer_match.py llm_critic.py  
├─ sandbox/      runner.py   limits.py  
├─ bench/        datasets/  runner.py  metrics.py  plots.py  mast_labels.py  
├─ web/          app.py     sse.py    templates/  static/  
├─ traces/       *.jsonl  
├─ configs/      conditions/{A,A_plus,B,C,D}.yaml  
└─ tests/
```

Conditions are **YAML configs over one engine**, not five codebases. Adding a baseline must cost a config file.

PART D — EXECUTION

D1. Three-person split with real interfaces

Contracts are frozen in the first two days so all three work in parallel without blocking.

Member	Owns	Interface they publish	Viva defence
Gautam (23BCS032) ~50%	Orchestrator loop, analyzer, planner, decision policy, budget governor , state + trace, provider abstraction + cache, FastAPI + SSE + UI, deployment, integration	RunState , WorkflowSpec , Budget , /api/run	"How does the system decide to add an agent, and how is budget held equal across conditions?"
Chirag (23BCS024) ~30%	Role library + prompt templates + versioning, Pydantic output schemas, LLM critic validator, Gemini + Groq integrations, context packaging, prompt ablations	Role , AgentOut schemas, llm_critic()	"Why this prompt structure, and what changed when you varied it?"
Aniket (23BCS015) ~20%	3 benchmark datasets, execution validators (pytest/SQL/answer), resumable experiment runner , metrics + plots, MAST failure labelling, reproducibility	Task , Verdict , run_suite() , metrics.csv	"How is correctness measured, and what are the failure categories?"

Aniket's slice is the smallest in volume but produces **every number in the results section** — worth saying out loud so it isn't treated as leftover work.

D2. Timeline (from 24 Sep — internal targets, not eval dates)

Window	Deliverable
24–28 Sep (Phase 0)	This package + full literature notes. Repo skeleton, contracts frozen , provider abstraction + cache, 5 smoke tasks running. No research code yet.
29 Sep – 10 Oct (Phase 1 → 50%)	Conditions A , A+ , B working end-to-end on the code family. Real LLM calls, pytest validation, refinement loop, termination, JSONL trace, UI v1 with live agent cards, first metrics table. Unseen runtime tasks must work .
10 Oct	Internal 50% gate . Freeze features 9 Oct; rehearse demo.
19–21 Oct	Mid-sem eval: motivation, literature, architecture, working loop, baseline setup, initial numbers, remaining plan.
11–31 Oct (Phase 2)	Adaptive planner + decision policy (condition D), condition C , SQL + math families, heterogeneous validators, second provider, budget governor enforcing matched spend.
1–10 Nov (Phase 3)	Full frozen sweep, metrics, plots, iteration-curve ablation (RQ4), validator ablation (RQ3), MAST labelling, demo hardening + replay mode, deploy, report.
10 Nov	Internal 100% . 13 days buffer.
23–27 Nov	Final evaluation.

Phase 1 deliberately excludes the adaptive part. Getting A/A+/B rock-solid first means the 50% gate is a working system, and condition D is then a contained addition rather than the thing everything depends on.

D3. Risks

Risk	Severity	Mitigation
Quota exhaustion mid-sweep	High	Cache + 3 keys + resumable runner + reserved demo key (C2)

Risk	Severity	Mitigation
Live demo dies in front of panel	High	Trace replay mode, built Phase 1; reserved key; pre-warmed cache
Adaptive (D) doesn't beat A+	Medium	This is a result, not a failure — the framing is built for it
Sandbox escape / hung subprocess	Medium	Hard rlimits + timeout; no arbitrary exec on public host
Scope creep into a framework	Medium	Custom core capped at ~400 lines; conditions are configs
Three-way coordination stalls	Medium	Contracts frozen in first 2 days; each owns a directory
Benchmark contamination	Low	Declared in threats to validity; EvalPlus tests reduce it

D4. Threats to validity (report section — write it honestly)

Model nondeterminism (mitigated by seeds + cache); benchmark contamination in HumanEval/GSM8K; single primary model family (Gemini) so results may not transfer; correlated errors when generator and critic share a model — which is exactly why the different-model reviewer comparison exists; small n per family (40) limits statistical power; free-tier latency confounds wall-clock measurements, so token count is the primary cost metric, not seconds.

FINAL ARCHITECTURE TO APPROVE

Title: *Adaptive Multi-Agent Orchestration under Matched Compute Budgets: An Empirical Study of When Multi-Agent Refinement Improves Task Correctness* **System name:** ARBITER — Adaptive Role-Based Iterative Task Execution and Refinement

Contribution (stated honestly): not a new architecture. A **controlled, budget-matched evaluation** of adaptive multi-agent orchestration across three task families of differing verifier strength, plus an open, reproducible research harness. We claim no novelty in multi-agent collaboration or in adaptivity.

Stack: Python 3.12 · FastAPI + SSE · Pydantic v2 · custom ~400-line orchestrator · Gemini Flash-Lite primary with Groq/OpenRouter secondary behind a provider abstraction · SQLite + JSONL · server-rendered HTML + htmx · Render free tier.

Scope: 120 frozen tasks across code / SQL / math. Five conditions (A, A+, B, C, D). Dynamic role **selection** from a fixed library — never runtime invention. Hard budget caps on tokens, calls, iterations and wall time. Local sandboxed execution; **no arbitrary code execution on the public host**.

Evaluation: RQ1–RQ4 as stated, measured from JSONL traces, failures classified with MAST, benchmark frozen before final runs.

Non-negotiables before any code: response cache, budget governor and resumable runner are Phase-0/1 components — the project fails on quota without them.

Coverage of the brief's 52 required items

1–7 → Context, A2, B1 · 8–10 → A1, A2 · 11–12 → A3, A4 · 13–16 → B5, B6 · 17–21 → B2, B3, B4 · 22–24 → B6, B7, B9 · 25–26 → B8, D3 · 27–30 → C1, C2 · 31–34 → C1, C3, B7 · 35–37 → B7, D2 Phase 3, D4 · 38–40 → B9, C1, C3 · 41–42 → D2 · 43–44 → D1 · 45 → C3 `tests/` + D2 · 46–48 → deferred: paper not a goal this semester (team decision) · 49 → D3 · 50–52 → Final Architecture + D2

Items 46–48 (paper structure, expected results, publication contribution) are intentionally out of scope: the team's stated priority is prototype + report + viva, not publication. The experimental design nonetheless remains paper-grade should that change.

Nothing has been created, installed or configured. Awaiting approval before Phase 0 implementation begins.